

A Evolução: Funções Tradicionais vs. Arrow Functions (Exercícios)

1. Conversão para Arrow Function:

```
const somar = (a, b) => a + b;
```

2. Boas-vindas:

```
const saudar = nome => "Olá, " + nome;
```

3. Sem parâmetros (Data atual):

```
const obterDataAtual = () => new Date();
```

4. Por que usar `const`?

Usamos `const` para garantir que a referência da função seja constante, impedindo que ela seja acidentalmente reatribuída ou substituída durante a execução do código.

5. Identificação do erro:

O erro é o uso de chaves `{}` sem a palavra-chave `return`; em Arrow Functions, quando você abre chaves, o **retorno implícito** é desativado, então o correto seria `x => x * x` ou `x => { return x * x; }`.

Resumo Visual das Diferenças

Para ajudar na memorização, veja como a sintaxe se simplifica:

- **Tradicional:** Exige as palavras `function` e `return`.
- **Arrow:** Se tiver apenas uma linha, o `return` e as `{}` são opcionais. Se tiver apenas um parâmetro, os `()` também são opcionais.

Higher-Order Functions: map, filter, reduce (Exercícios)

1. Dobro dos valores com `map`

JavaScript

```
const numeros = [1, 2, 3, 4];  
const dobro = numeros.map(n => n * 2);  
// Resultado: [2, 4, 6, 8]
```

2. Filtrar números pares com `filter`

JavaScript

```
const lista = [10, 15, 20, 25];  
const pares = lista.filter(n => n % 2 === 0);  
// Resultado: [10, 20]
```

3. Produto total com `reduce`

JavaScript

```
const fatores = [2, 3, 4];  
const produto = fatores.reduce((acumulador, n) => acumulador * n, 1);  
// Resultado: 24 (2 * 3 * 4)  
// Nota: Usamos 1 como valor inicial porque qualquer número vezes 0 é 0.
```

4. Encadeamento (filter + map)

JavaScript

```
const pessoas = [
  { nome: "Ana", idade: 17 },
  { nome: "Beto", idade: 25 },
  { nome: "Carla", idade: 30 }
];

const cumprimentos = pessoas
  .filter(p => p.idade >= 18) // Primeiro, separa os adultos
  .map(p => p.nome.length); // Depois, transforma em números (comprimento do nome)
// Resultado: [4, 5] (Beto e Carla)
```

5. O papel do número 0 no reduce

O número 0 é o **valor inicial do acumulador**. Na nossa analogia da fazenda, ele representa o **tanque reservatório começando vazio**. Sem ele, o `reduce` usaria o primeiro item da lista como valor inicial, o que poderia causar erros se a lista estivesse vazia ou se os itens fossem objetos em vez de números.

Resumo Visual do Encadeamento

Quando você junta os métodos (como no exercício 4), os dados fluem como em uma linha de montagem:

- **Entrada:** Dados brutos.
- **Filter:** Remove o que não serve (reduz o tamanho da lista).
- **Map:** Transforma o que sobrou (muda o formato dos dados).
- **Saída:** Uma nova lista pronta para uso.

Spread Operator e Destructuring (Desestruturação) (Exercícios)

1. Desestruturação de Array

JavaScript

```
const notas = [8, 9, 10];
const [nota1, nota2] = notas;
// nota1 = 8, nota2 = 9
```

2. Alias (Apelido) em Objeto

JavaScript

```
const usuario = { id: 101, nome: "Gemini" };
const { id: codigo } = usuario;
// A variável criada se chama 'codigo' e vale 101.
```

3. Clonar e Adicionar (Spread em Array)

JavaScript

```
const a = [1, 2];
const b = [...a, 3];
// b = [1, 2, 3]
```

4. Mesclear Objetos (Spread em Objeto)

JavaScript

```
const obj1 = { a: 1 };
const obj2 = { b: 2 };
const novoObj = { ...obj1, ...obj2 };
// novoObj = { a: 1, b: 2 }
```

5. Por que não alterar diretamente?

Alterar `usuarioAtualizado.cargo` diretamente viola o princípio da **Imutabilidade**. Nas boas práticas funcionais, você nunca deve modificar o objeto original (causar um "efeito colateral"), mas sim criar uma nova cópia com a alteração desejada. Isso evita bugs imprevisíveis em outras partes do código que também usem aquele objeto e facilita o rastreamento de mudanças no estado da aplicação.

Analogia Visual:

- O Spread Operator: Pense no **Spread Operator (...)** como o ato de tirar os itens de dentro de uma caixa e espalhá-los em uma nova. O **Spread** "espalha" os itens para fora da estrutura original.
- A **Desestruturação** é como ter um kit de ferramentas e pegar apenas as chaves que você vai usar, deixando o resto na caixa. A **Desestruturação** "extraí" itens específicos para variáveis independentes.

Funções Puras e Escopo de Variáveis (Exercícios)

1. `gerarId` é Pura ou Impura?

Ela é **Impura**. Uma função pura deve retornar o mesmo resultado para os mesmos argumentos. Como `Math.random()` gera um valor diferente a cada execução, o resultado é imprevisível. Além disso, ela depende de um estado externo (o gerador de números aleatórios do sistema).

2. `let` vs `var` dentro de um `if`

A principal diferença é o **escopo**:

- **let**: Tem escopo de **bloco**. Se declarada dentro do `if`, ela "morre" ali dentro e não pode ser acessada do lado de fora.
- **var**: Tem escopo de **função**. Mesmo declarada dentro de um `if`, ela sofre *hoisting* (içamento) e fica disponível em qualquer lugar da função onde o `if` está inserido.

3. Transformando em Função Pura

- **Impura (depende de global):**

JavaScript

```
const taxa = 5;
const calcularTotal = (preco) => preco + taxa;
```

- **Pura (parâmetro explícito):**

JavaScript

```
const calcularTotalPuro = (preco, taxa) => preco + taxa;
```

Agora, o resultado depende apenas do que foi passado para ela.

4. Testes Unitários e Funções Puras

Funções puras são mais fáceis de testar porque são **previsíveis e isoladas**. Como elas não dependem de bancos de dados, APIs ou variáveis globais, você não precisa configurar "cenários complexos" (mocks/stubs). Basta passar uma entrada e verificar se a saída é a esperada.

5. Relação de map com Funções Puras

O método `map` foi projetado para trabalhar com funções puras. Ele recebe uma função de transformação que:

1. Não altera os dados originais (imutabilidade).
 2. Retorna um novo valor baseado apenas no item atual da lista.
- Isso garante que o processamento da lista seja seguro e livre de efeitos colaterais indesejados.

Visualizando o Escopo: var vs let

Para nunca mais esquecer a diferença de "vazamento" de memória entre os dois:

O Fluxo da Função Pura: Lembre-se da balança, a entrada entra, a lógica processa, e a saída sai, sem mexer em nada ao redor.

Exercícios de Fixação

1. Limpador de Estoque (`filter`)

JavaScript

```
const estoque = [
  { nome: 'Teclado', quantidade: 10 },
  { nome: 'Mouse', quantidade: 0 },
  { nome: 'Monitor', quantidade: 5 },
  { nome: 'Cabo HDMI', quantidade: 0 }
];

const itensDisponiveis = estoque.filter(produto => produto.quantidade > 0);

console.log("Itens em estoque:", itensDisponiveis);
```

2. Inflação Aplicada (`map`)

JavaScript

```
const precosOriginais = [100, 200, 50, 10];

const aplicarInflacao = precos => precos.map(preco => preco * 1.05);

const precosAjustados = aplicarInflacao(precosOriginais);
console.log("Preços com +5%:", precosAjustados);
```

3. Desempacotador de Usuários (Deep Destructuring)

JavaScript

```
const cliente = {
  perfil: { nome: 'Luiz', idade: 25 },
  contato: { email: 'l@email.com', fone: '999' }
};
```

```
// Extração profunda direta
const { perfil: { nome }, contato: { email } } = cliente;

console.log(nome, email); // Saída: Luiz l@email.com
```

4. Faturamento do Dia (reduce)

```
JavaScript
const vendasDia = [{venda: 1, valor: 45.5}, {venda: 2, valor: 120.0}];

const faturamentoTotal = vendasDia.reduce((total, atual) => total + atual.valor, 0);

console.log(`Faturamento Total: R$ ${faturamentoTotal.toFixed(2)}`);
```

5. O Clone Seguro (Spread Operator)

```
JavaScript
const usuario = { id: 1, plano: 'Basic' };

const usuarioPremium = {
  ...usuario,
  plano: 'Pro',
  upgradeDate: new Date()
};

console.log("Original:", usuario);
console.log("Premium:", usuarioPremium);
console.log("São independentes?", usuario !== usuarioPremium);
```

6. Relatório de E-commerce Múltiplo (Chaining)

```
JavaScript
const vendasMassa = [
  { status: 'APROVADO', produtos: [{ valor: 100 }, { valor: 50 }] },
  { status: 'CANCELADO', produtos: [{ valor: 200 }] },
  { status: 'APROVADO', produtos: [{ valor: 30 }, { valor: 20 }] }
];

const totalFaturado = vendasMassa
  .filter(v => v.status === 'APROVADO')
  .map(v => v.produtos.reduce((sub, p) => sub + p.valor, 0))
  .reduce((total, subtotal) => total + subtotal, 0);

console.log("Faturamento Total Empresa: R$", totalFaturado);
```

7. Sanitizar e Agrupar Dados

```
JavaScript
const dadosSujos = ["Ana", null, "João", "", "Ana", "Bia", "João", "Ana"];

const nomesAgrupados = dadosSujos
```

```
.filter(nome => nome) // Remove null, undefined e strings vazias
.reduce((acc, nome) => {
  acc[nome] = (acc[nome] || 0) + 1;
  return acc;
}, {});
```

```
console.log("Dados Agrupados:", nomesAgrupados);
```

8. Refatoração Imperativa para Declarativa

O Ruim (Imperativo):

JavaScript

```
const alunos = [{ nome: 'Nico', nota: 8 }, { nome: 'Duda', nota: 6 }, { nome: 'Guto', nota: 9 }];
let aprovadosNomes = [];
```

```
for (let i = 0; i < alunos.length; i++) {
  if (alunos[i].nota > 7) {
    aprovadosNomes.push(alunos[i].nome.toUpperCase());
  }
}
console.log(aprovadosNomes);
```

O Bom (Declarativo):

JavaScript

```
const aprovadosDeclarativo = alunos
  .filter(a => a.nota > 7)
  .map(a => a.nome.toUpperCase());
```

```
console.log(aprovadosDeclarativo);
```

9. Função Pura de Carrinho de Compras

JavaScript

```
const carrinhoAtual = [{ id: 1, item: 'Meia' }];
const novoItem = { id: 2, item: 'Tênis' };
```

```
const adicionarAoCarrinho = (carrinho, item) => [...carrinho, item];
```

```
const novoCarrinho = adicionarAoCarrinho(carrinhoAtual, novoItem);
```

```
console.log("Novo Carrinho:", novoCarrinho);
console.log("Original intacto?", carrinhoAtual.length === 1);
```

10. Utilitários de Dados Genéricos

Arquivo utils.js:

JavaScript

```
export const obterApenasAtivos = (arr) => arr.filter(item => item.ativo);

export const formatarPrecos = (arr) => arr.map(p => `R$ ${p.toFixed(2)}`);
```

```
export const somarCampo = (arr, campo) => arr.reduce((acc, obj) => acc + obj[campo], 0);
```

```
export const clonarComData = (obj) => ({ ...obj, dataProcessamento: new Date() });
```

Uso no arquivo central:

JavaScript

```
import * as Utils from './utils.js';
```

```
const usuarios = [{ nome: 'A', ativo: true }, { nome: 'B', ativo: false }];
```

```
const produtos = [{ preco: 10.5 }, { preco: 20.0 }];
```

```
console.log("Ativos:", Utils.obterApenasAtivos(usuarios));
```

```
console.log("Formatados:", Utils.formatarPrecos([10, 20.5, 30]));
```

```
console.log("Soma de Preços:", Utils.somarCampo(produtos, 'preco'));
```

```
console.log("Clone:", Utils.clonarComData({ id: 1 }));
```